

Competitive Programming

Section 33: Inclusion-Exclusion

Competitive Programming Course

2026-07-06

Table of contents

1	Why inclusion-exclusion exists	3
2	General formula	4
3	Why the signs work	5
4	Pattern 1: union of divisibility sets	6
5	Pattern 2: all symbols appear	7
6	Pattern 3: bounded stars and bars	8
7	Implementation notes	9
8	Common mistakes	10
9	Submission checklist	11

1 Why inclusion-exclusion exists

When sets overlap, adding their sizes double-counts intersections.

For two sets:

$$|A \text{ union } B| = |A| + |B| - |A \text{ intersection } B|$$

For three sets:

$$\begin{aligned} &|A \text{ union } B \text{ union } C| \\ &= \text{singles} - \text{pairs} + \text{triples} \end{aligned}$$

This alternating pattern is inclusion-exclusion.

2 General formula

For events $E_1, E_2, \dots, E_m$, the size of their union is:

```
sum over nonempty subsets S:  
   $(-1)^{|S|+1} * \text{size}(\text{intersection of events in } S)$ 
```

Odd subset sizes are added. Even subset sizes are subtracted.

3 Why the signs work

Suppose an object belongs to exactly t events. Its total contribution is:

$$C(t,1) - C(t,2) + C(t,3) - \dots +/ - C(t,t)$$

This sum equals 1, so every object in at least one event is counted once. An object in zero events is never included.

4 Pattern 1: union of divisibility sets

Let:

```
Ei = numbers ≤ N divisible by d[i]
```

Intersection of a subset means numbers divisible by every selected divisor, which is the same as divisible by their lcm.

```
count(intersection) = floor(N / lcm(selected divisors))
```

Use subset enumeration when $m \leq 20$.

5 Pattern 2: all symbols appear

Count strings of length n over k symbols where every symbol appears.

Total strings:

$$k^n$$

Bad event E_i : symbol i is missing.

If a set of j symbols is missing, there are:

$$(k-j)^n$$

strings. Choose which j symbols are missing:

$$C(k, j)$$

Answer:

$$\sum_{j=0..k} (-1)^j * C(k, j) * (k-j)^n$$

6 Pattern 3: bounded stars and bars

Count:

$$\begin{aligned}x_1 + \dots + x_n &= s \\ 0 \leq x_i &\leq b\end{aligned}$$

Without upper bounds, Section 32 gives:

$$C(s+n-1, n-1)$$

Bad event E_i : child i gets at least $b+1$ candies.

If j chosen children are bad, give each of them $b+1$ candies first. Remaining candies:

$$s - j(b+1)$$

Then distribute the rest freely:

$$C(s - j(b+1) + n - 1, n - 1)$$

Answer:

$$\sum_{j \geq 0} (-1)^j * C(n, j) * C(s - j(b+1) + n - 1, n - 1)$$

only while $s - j(b+1) \geq 0$.

7 Implementation notes

For subset inclusion-exclusion:

```
for mask in 1..(1<<m)-1:
    bits = popcount(mask)
    contribution = intersection size for selected events
    add if bits odd, subtract if bits even
```

For formula inclusion-exclusion, loop over the number of bad events j .

For modular subtraction:

```
ans = (ans - term + MOD) % MOD;
```

Python's final `ans % MOD` normalizes negative values.

8 Common mistakes

Subtracting odd subsets instead of even subsets.
Counting "all events happen" when the problem asks "at least one event happens."
Forgetting lcm in divisibility intersections.
Letting lcm overflow before comparing with N.
In bounded stars-and-bars, subtracting b instead of b+1.
Continuing the loop after the remaining sum becomes negative.

9 Submission checklist

1. What are the events?
2. Are you counting a union of good events or excluding bad events?
3. What is the intersection size for a chosen subset?
4. Is the sign based on subset size?
5. Are modular subtractions normalized?